

Laboratorio: Optimización por Colonia de Hormigas

INTEGRANTES:

Johan Steven Garcia Niño

Cesar Alberto Mora Duarte

1. Introducción

Este laboratorio implementa el algoritmo ACO (Ant Colony Optimization) para resolver un problema de búsqueda de caminos en un mapa 2D con obstáculos. La característica principal es que todas las hormigas se procesan en paralelo usando la GPU, lo que acelera significativamente la simulación respecto a una implementación tradicional en CPU.

El problema a resolver es simple de entender: dado un mapa con paredes, una colonia de hormigas debe encontrar el camino más corto desde su punto de inicio (colonia) hasta la comida, aprendiendo iterativamente cuál es la mejor ruta.

2. Estructura del Notebook

El notebook está organizado en 6 celdas, cada una con una responsabilidad clara:

Celda 1 – Configuración del entorno

Detecta si hay una GPU disponible. Si CuPy (la librería GPU) está instalada, la usa; si no, cae back a NumPy con la misma API. Esto hace que el código sea portable: corre igual en CPU o GPU sin cambiar nada.

Celda 2 – Definición del mapa

El mapa es una matriz de 11×15 con cuatro tipos de celdas:

- 0 → celda libre (la hormiga puede pasar)
- 1 → pared (bloqueada)
- 2 → colonia, el punto de inicio en (9,1)

- 3 → comida, el destino en (1,13)

El mapa tiene 11 celdas libres bloqueadas por paredes que obligan a las hormigas a explorar distintas rutas.

Celda 3 – Clase ACO

El núcleo del algoritmo. Implementa el movimiento vectorizado de todas las hormigas en paralelo sobre la GPU. El proceso por iteración es:

- Calcular las 8 celdas vecinas posibles para cada hormiga (conectividad diagonal incluida).
- Filtrar las inválidas: fuera de límites, paredes, o ya visitadas.
- Calcular el score de cada dirección válida: $\tau^\alpha \cdot \eta^\beta$ (feromonas $\times$ heurística).
- Seleccionar la dirección usando el truco Gumbel-max, que es equivalente a un muestreo probabilístico pero completamente vectorizado en GPU.
- Actualizar posiciones, marcar visitadas, detectar llegada a la meta.
- Al final: evaporar feromonas y depositar nuevas en los caminos exitosos.

Celda 4 – Visualización

Se genera el canvas de visualización de la malla en donde se visualizaran los movimientos de las hormigas y el camino de feromonas con su evaporación correspondiente.

Celda 5 – Ejecución principal

Instancia la clase ACO con los parámetros definidos, corre las M iteraciones y anima el progreso en tiempo real dentro del notebook.

Celda 6 – Benchmark CPU vs GPU

Ejecuta la misma simulación en CPU (NumPy) y en GPU (CuPy) para la comparación de simulaciones.

3. Parámetros del Algoritmo

Los parámetros usados en la ejecución principal son:

Parámetro	Valor	Descripción
N = 30	30	Número de hormigas que se mueven en paralelo cada iteración.
M = 150	150	Total de iteraciones del algoritmo.
alpha (α) = 1.0	1.0	Peso de las feromonas en la decisión de movimiento.
beta (β) = 3.0	3.0	Peso de la heurística (distancia al destino). Más alto = más greedy.
rho (ρ) = 0.1	0.1	Tasa de evaporación por iteración. $\tau = (1 - \rho) \cdot \tau$
tau_init = 1.0	1.0	Nivel inicial de feromonas en todas las celdas libres.

```

"""
ACO vectorizado: todas las N hormigas se mueven en paralelo cada tick.

Parámetros
-----
N      : cantidad de hormigas
M      : iteraciones totales
alpha  : peso de feromonas ( $\tau^\alpha$ )
beta   : peso de heurística ( $\eta^\beta$ )
rho     : tasa de evaporación  $\rho$ 
tau_init : nivel inicial de feromonas  $\tau_0$ 
max_steps: pasos máximos por hormiga por iteración
"""

```

El beta alto (3.0) hace que las hormigas privilegien avanzar hacia la comida sobre seguir feromonas. Esto acelera la convergencia inicial pero puede hacer que se pierdan atajos no obvios.

4. Aspectos Técnicos Clave

4.1 Vectorización en GPU

El punto más importante del laboratorio es que las N hormigas se procesan simultáneamente en cada paso, no una por una. Todas las operaciones (calcular

vecinos, scores, muestreo) trabajan sobre arrays de shape (N, 8), donde cada fila corresponde a una hormiga y cada columna a una de las 8 direcciones posibles. Esto se ejecuta como una sola operación matricial en la GPU.

4.2 Gumbel-max Trick

La selección de dirección no es un argmax directo (eso sería greedy puro), sino un muestreo probabilístico. El truco Gumbel consiste en añadir ruido Gumbel(0,1) al logaritmo del score y luego tomar el argmax. Matemáticamente es equivalente a samplear de una distribución Categorical con probabilidades proporcionales al score, pero es paralelizable sin necesidad de acumular sumas de probabilidades. Esto permite que el muestreo de las N hormigas se haga en una sola operación GPU.

4.3 Depósito de Feromonas

Se usa `cp.add.at()` para depositar feromonas en las celdas del camino de cada hormiga exitosa. El monto depositado es $1/\text{longitud_camino}$, lo que premia naturalmente los caminos más cortos.

4.4 Transferencia GPU → CPU

El historial de posiciones se guarda en GPU durante toda la iteración. Solo al terminar se transfiere a CPU para reconstruir los caminos. Esto minimiza el cuello de botella de memoria entre GPU y CPU, que es uno de los problemas más comunes en programación GPU.

5. Resultados Esperados

Al ejecutar el laboratorio se espera que:

- En las primeras iteraciones las hormigas exploran aleatoriamente y pocas llegan a la meta.
- Con el tiempo las feromonas se concentran en los caminos más eficientes y más hormigas llegan exitosamente.
- El mejor camino converge hacia la ruta más corta posible en el mapa dado.
- En el benchmark, la GPU muestra speedup solo si el grid y el número de hormigas son suficientemente grandes para justificar el overhead de CuPy.

En el benchmark de la celda 6 (N=50, M=80), el resultado puede variar: en Google Colab con T4 la GPU gana, pero en un grid 11×15 con pocas hormigas el overhead de

CuPy puede igualar o superar el beneficio. Esto es un resultado pedagógico importante: la GPU no siempre es más rápida.

6. Conclusiones

Este laboratorio ilustra cómo un algoritmo bioinspirado clásico puede adaptarse para aprovechar el paralelismo masivo de las GPUs modernas. Los puntos más importantes son:

- ACO es naturalmente paralelo: cada hormiga es independiente dentro de una iteración.
- La vectorización con CuPy permite procesar cientos de hormigas simultáneamente sin bucles Python.
- El truco Gumbel-max es una técnica elegante para hacer muestreo probabilístico de forma vectorizada.
- El overhead GPU es real: para problemas pequeños, CPU puede ser más eficiente.
- El diseño CPU/GPU intercambiable (usando cp como alias de NumPy o CuPy) es una buena práctica de ingeniería para código portable.

La aceleración real de GPU se justifica cuando el problema escala: grids de 100×100 con 200+ hormigas pueden mostrar speedups de 10x o más sobre la implementación CPU equivalente.

Resultados:



